

# CondVar con Deadline

---

- Esempio variabile di condizione con deadline

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#include <errno.h>
```

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int ready = 0; // evento da attendere
```

```
// questo thread simula qualcuno che *forse* segnala dopo un po'
void* worker(void* arg) {
    // cambiare per verificare timeout
    sleep(3);
    pthread_mutex_lock(&mtx);
    ready = 1;
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&mtx);
    return NULL;
}
```

# CondVar con Deadline

---

- Esempio variabile di condizione con deadline

```
int main(void) {
    pthread_t th;
    pthread_create(&th, NULL, worker, NULL);

    pthread_mutex_lock(&mtx);

    // calcola la deadline: ora + 2 secondi
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    ts.tv_sec += 2; // deadline tra 2 secondi

    int rc = 0;
    while (!ready && rc == 0) {
        rc = pthread_cond_timedwait(&cond, &mtx, &ts);
        // ritorna 0 se qualcuno ha fatto signal/broadcast
        // ritorna ETIMEDOUT se è scaduta la deadline
    }

    if (ready) {
        printf("Entro la deadline\n");
    } else if (rc == ETIMEDOUT) {
        printf("Timeout scaduto\n");
    } else {
        printf("Error condtimefwait:%d\n", rc);
    }
    pthread_mutex_unlock(&mtx);
    pthread_join(th, NULL);
    return 0;
}
```

# Classici Problemi di Sincronizzazione

---

- Problemi classici usati per testare schemi di sincronizzazione
  - Bounded-Buffer Problem
  - Readers and Writers Problem
  - Dining-Philosophers Problem

# Bounded-Buffer Problem

---

- Buffer di  $n$  slot, ognuno può contenere un articolo
- Il buffer può essere pieno quindi il produttore deve attendere
- Il buffer può essere vuoto quindi il consumatore deve attendere
- Si parte con il buffer vuoto
- $M$  produttori riempiono il buffer
- $N$  consumatori lo svuotano

# Bounded-Buffer Problem

---

- Buffer di ***n*** slot, ognuno può contenere un articolo
- Il buffer può essere pieno quindi il produttore deve attendere
- Il buffer può essere vuoto quindi il consumatore deve attendere
- Si parte con il buffer vuoto
- M produttori riempiono il buffer
- N consumatori lo svuotano
  
- Serve:
  - Uno stato condiviso (buffer e contatori)
  - Un mutex per proteggerlo
  - Due cond var: **not\_full**, **not\_empty**

# Bounded-Buffer Problem

---

- Buffer di ***n*** slot, ognuno può contenere un articolo
- Struttura condivisa:

```
#define BUF_SIZE 10

typedef struct {
    int buf[BUF_SIZE];
    int in;    // prossima posizione dove scrivere
    int out;   // prossima posizione da leggere
    int count; // quanti elementi ci sono

    pthread_mutex_t mtx;
    pthread_cond_t not_full;
    pthread_cond_t not_empty;
} bbuff_t;
```

- mtx protegge l'accesso alla struttura
- $0 \leq \text{count} \leq \text{BUF\_size}$
- $\text{in/out} \text{ in mod } \text{BUF\_SIZE}$

# Bounded-Buffer Problem

---

- Buffer di ***n*** slot, ognuno può contenere un articolo
- Funzione di inizializzazione della struttura:

```
void bbuff_init(bbuff_t *b) {
    b->in = 0;
    b->out = 0;
    b->count = 0;

    // inizializzazione runtime
    pthread_mutex_init(&b->mtx, NULL);
    pthread_cond_init(&b->not_full, NULL);
    pthread_cond_init(&b->not_empty, NULL);
}

int main(void) {
    bbuff_t *bb = malloc(sizeof(bbuff_t));
    bbuff_init(bb);
    // ... codice
    bbuff_destroy(bb);
    free(bb);
}
```

# Bounded-Buffer Problem

---

- Buffer di ***n*** slot, ognuno può contenere un articolo
- Funzione di inizializzazione della struttura:

```
void bbuff_destroy(bbuff_t *b) {  
    pthread_mutex_destroy(&b->mtx);  
    pthread_cond_destroy(&b->not_full);  
    pthread_cond_destroy(&b->not_empty);  
}
```

```
int main(void) {  
    bbuff_t *bb = malloc(sizeof(bbuff_t));  
    bbuff_init(bb);  
    // ... codice  
    bbuff_destroy(bb);  
    free(bb);  
}
```



# Bounded-Buffer Problem

---

- Buffer di ***n*** slot, ognuno può contenere un articolo
- Funzione di distruzione della struttura:

```
void bbuff_destroy(bbuff_t *b) {  
    pthread_mutex_destroy(&b->mtx);  
    pthread_cond_destroy(&b->not_full);  
    pthread_cond_destroy(&b->not_empty);  
}
```

```
int main(void) {  
    bbuff_t *bb = malloc(sizeof(bbuff_t));  
    bbuff_init(bb);  
    // ... codice  
    bbuff_destroy(bb);  
    free(bb);  
}
```

# Bounded-Buffer Problem

---

- Buffer di  $n$  slot, ognuno può contenere un articolo
- Struttura del main:

```
int main(void) {  
    bbuff_t *bb = malloc(sizeof(bbuff_t));  
    if (!bb) { perror("malloc"); exit(1); }  
  
    bbuff_init(bb);  
  
    // avvio thread producer/consumer, ecc.  
  
    // join dei thread  
    // ...  
  
    bbuff_destroy(bb);  
    free(bb);  
  
    return 0;  
}
```

# Bounded-Buffer Problem

---

- Buffer di ***n*** slot, ognuno può contenere un articolo
- Lancio e attesa dei produttori/consumatori:

```
pthread_t prod[N_PROD], cons[N_CONS];

// avvio producer
for (int i = 0; i < N_PROD; i++)
    pthread_create(&prod[i], NULL, producer, bb);

// avvio consumer
for (int i = 0; i < N_CONS; i++)
    pthread_create(&cons[i], NULL, consumer, bb);

// aspetto che i producer finiscano
for (int i = 0; i < N_PROD; i++)
    pthread_join(prod[i], NULL);

//Uccido I consumatori con un simbolo di terminazione
for (int i = 0; i < N_CONS; i++)
    put_item(bb, KILL_PILL);

// aspetto che i consumer finiscano
for (int i = 0; i < N_CONS; i++)
    pthread_join(cons[i], NULL);
```

# Bounded-Buffer Problem

---

- Buffer di ***n*** slot, ognuno può contenere un articolo
- Codice produttore:

```
/* --- Producer --- */
void* producer(void* arg) {
    bbuff_t *b = (bbuff_t*)arg;
    for (int i = 0; i < N_ITEMS; i++) {
        put_item(b, i);           // produce solo il numero i
        printf("[P] produced %d\n", i);
        usleep(10000);           // solo per rallentare un po'
    }
    return NULL;
}
```

# Bounded-Buffer Problem

---

- Buffer di ***n*** slot, ognuno può contenere un articolo
- Codice consumatore:

```
/* --- Consumer --- */
void* consumer(void* arg) {
    bbuff_t *b = (bbuff_t*)arg;
    for (;;) {
        int item = get_item(b);
        if (item == KILL_PILL) {
            printf(" [C] received KILL_PILL — exiting\n");
            break;
        }
        printf(" [C] consumed %d\n", item);
        usleep(15000);
    }
    return NULL;
}
```

# Bounded-Buffer Problem

---

- Buffer di ***n*** slot, ognuno può contenere un articolo
- Soluzione:

```
void put_item(bbuff_t *b, int item) {  
    pthread_mutex_lock(&b->mtx);  
  
    // Se il buffer è pieno, aspetta che ci sia spazio  
    while (b->count == BUF_SIZE)  
        pthread_cond_wait(&b->not_full, &b->mtx);  
  
    // Inserisci l'elemento nel buffer circolare  
    b->buf[b->in] = item;  
    b->in = (b->in + 1) % BUF_SIZE;  
    b->count++;  
  
    // Ora il buffer non è più vuoto → sveglia un eventuale consumer  
    pthread_cond_signal(&b->not_empty);  
  
    pthread_mutex_unlock(&b->mtx);  
}
```

# Bounded-Buffer Problem

---

- Buffer di  $n$  slot, ognuno può contenere un articolo
- Soluzione:

```
int get_item(bbuff_t *b) {
    pthread_mutex_lock(&b->mtx);

    // Se il buffer è vuoto, aspetta che arrivi qualcosa
    while (b->count == 0)
        pthread_cond_wait(&b->not_empty, &b->mtx);

    // Preleva un elemento dal buffer circolare
    int item = b->buf[b->out];
    b->out = (b->out + 1) % BUF_SIZE;
    b->count--;

    // Ora c'è almeno uno spazio libero → sveglia un eventuale
    producer
    pthread_cond_signal(&b->not_full);

    pthread_mutex_unlock(&b->mtx);

    return item;
}
```

# Semafori

---

- Strumenti di sincronizzazione più sofisticati (dei mutex locks) per sincronizzare processi.
- Semaforo **S** – variable intera
- Modificata con due operazioni indivisibili (atomiche)
  - **wait()** e **signal()**
    - Originariamente detti **P()** e **V()** da Dijkstra (Proberen: to test, Verhogen: to increment)
- Definizione di **wait()**

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

- Definizione di **signal()**

```
signal(S) {  
    S++;  
}
```

## Operazioni Atomiche:

- Nessun altro processo può modificare il valore mentre sono in esecuzione
- Nel caso di wait sia test che decremento senza interruzioni



# Semafori Uso

---

- Distinzione tra due tipi di semafori:
- **Semaforo contatore** – valore intero varia su un dominio non ristretto
- **Semaforo binario** – valore intero varia tra 0 e 1
  - Come il **mutex lock**
- Controllo accesso a numero di risorse pari al valore inizializzato
- Può risolvere diversi problemi di sincronizzazione
- Esempio – vincoli di scheduling
  - Considera  $P_1$  e  $P_2$  con  $P_1$  che richiede  $S_1$  prima di  $S_2$

Crea un semaforo “**synch**” inizializzato a 0

**P1 :**

$S_1 ;$

**signal (synch) ;**

Thread join

Thread exit

Synch = 0

**P2 :**

**wait (synch) ;**

wait(S)

signal(S)

$S_2 ;$

- Può implementare un semaforo contatore **S** con un semaforo binario

# Produttori Consumatori

---

```
int n;  
semaphore mutex = 1;  
semaphore empty = n;  
semaphore full = 0
```

Due semafori per vincoli di scheduling  
tra produttore e consumatore + un lock

```
while (true) {  
    . . .  
    /* produce an item in next_produced */  
    . . .  
    wait(empty);  
    wait(mutex);  
    . . .  
    /* add next_produced to the buffer */  
    . . .  
    signal(mutex);  
    signal(full);  
}
```

```
while (true) {  
    wait(full);  
    wait(mutex);  
    . . .  
    /* remove an item from buffer to next_consumed */  
    . . .  
    signal(mutex);  
    signal(empty);  
    . . .  
    /* consume the item in next_consumed */  
    . . .  
}
```

# Semafori POSIX

---

- Nei sistemi POSIX (Linux, macOS, Unix), questa astrazione è implementata dalla **libreria <semaphore.h>**, che ci fornisce funzioni per creare e gestire semafori reali, utilizzabili nei programmi C con i thread (pthread).

```
#include <semaphore.h>
```

```
sem_t sem; // dichiarazione
```

- gestiti interamente in memoria del processo;
- tipici per sincronizzare thread (come nel bounded buffer)

# Semafori POSIX

---

- Nei sistemi POSIX (Linux, macOS, Unix), questa astrazione è implementata dalla **libreria <semaphore.h>**, che ci fornisce funzioni per creare e gestire semafori reali, utilizzabili nei programmi C con i thread (pthread).

| Funzione                                                                      | Significato                                                                                                                                   | Analogia teorica               |
|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| <pre>int sem_init(sem_t *sem,<br/>int pshared, unsigned int<br/>value);</pre> | Inizializza il semaforo con<br>valore iniziale <code>value</code> .<br><code>pshared = 0</code> se usato tra<br>thread dello stesso processo. | inizializza $S = \text{value}$ |
| <pre>int sem_wait(sem_t *sem);</pre>                                          | Decrementa $S$ se $S > 0$ ;<br>altrimenti blocca il thread<br>finché non lo diventa.                                                          | $\text{wait}(S)$ o $P(S)$      |
| <pre>int sem_post(sem_t *sem);</pre>                                          | Incrementa $S$ e sveglia<br>eventualmente un thread<br>bloccato.                                                                              | $\text{signal}(S)$ o $V(S)$    |
| <pre>int sem_destroy(sem_t *sem);</pre>                                       | Libera le risorse associate al<br>semaforo.                                                                                                   | distruzione                    |

# Semafori

---

- Produttori consumatori con semafori

```
#define BUF_SIZE 8
#define N_PROD  2
#define N_CONS  2
#define N_ITEMS 10
#define KILL_PILL -1 // valore speciale per fermare i consumer

/* ---- buffer circolare condiviso ---- */
int buffer[BUF_SIZE];
int in_idx = 0; // prossima posizione di scrittura
int out_idx = 0; // prossima posizione di lettura

/* ---- semafori ---- */
sem_t empty; // conta gli slot liberi      (parte da BUF_SIZE)
sem_t full;  // conta gli elementi disponibili (parte da 0)
sem_t mutex; // per proteggere buffer/in_idx/out_idx (parte da 1)
```

# Semafori

---

- Produttori consumatori con semafori

```
int main(void) {
    pthread_t prod[N_PROD], cons[N_CONS];

    /* inizializzo i semafori
    pshared = 0 → tra thread dello stesso processo */
    sem_init(&empty, 0, BUF_SIZE); // buffer inizialmente vuoto
    sem_init(&full, 0, 0);          // nessun elemento da consumare
    sem_init(&mutex, 0, 1);         // semaforo binario (come un mutex)

    /* creo i produttori */
    ...
    /* creo i consumatori */
    ...
    /* aspetto che i produttori finiscano */
    ...
    /* i produttori hanno finito: metto UNA KILL_PILL per ogni consumer */
    ...
    /* aspetto che i consumer terminino */
    ...
    /* distruggo i semafori */
    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}
```

# Semafori

---

- Produttori consumatori con semafori

```
/* ---- thread produttore ---- */
void* producer(void* arg) {
    for (int i = 0; i < N_ITEMS; i++) {
        put_item(i);           // produce solo i numeri 0..N_ITEMS-1
        printf("[P] produced %d\n", i);
        usleep(10000);
    }
    return NULL;
}

/* ---- thread consumatore ---- */
void* consumer(void* arg) {
    long id = (long)arg;
    for (;;) {
        int item = get_item();
        if (item == KILL_PILL) {
            printf(" [C%d] received KILL_PILL — exiting\n", id);
            break;
        }
        printf(" [C%d] consumed %d\n", id, item);
        usleep(15000);
    }
    return NULL;
}
```

# Semafori

---

- Produttore con semafori

```
/* inserisce un elemento nel buffer */
void put_item(int item) {
    /* aspetta che ci sia spazio */
    sem_wait(&empty);

    /* sezione critica */
    sem_wait(&mutex);
    buffer[in_idx] = item;
    in_idx = (in_idx + 1) % BUF_SIZE;
    sem_post(&mutex);

    /* adesso c'è un elemento in più */
    sem_post(&full);
}
```



# Semafori

---

- Consumatore con semafori

```
/* preleva un elemento dal buffer */
int get_item(void) {
    /* aspetta che ci sia almeno un elemento */
    sem_wait(&full);

    /* sezione critica */
    sem_wait(&mutex);
    int item = buffer[out_idx];
    out_idx = (out_idx + 1) % BUF_SIZE;
    sem_post(&mutex);

    /* adesso c'è uno slot libero in più */
    sem_post(&empty);

    return item;
}
```

# Semafori

---

- Notare:
  - Non c'è while come con `pthread_cond_wait`, il semaforo tiene il conto
  - `empty` e `full` sostituiscono il count del buffer
  - `mutex` serve lo stesso: i semafori contano gli elementi, ma non proteggono le variabili `in_idx` e `out_idx` → per quello serve il semaforo binario.

# Semafori

---

Un ponte può contenere al massimo  $N$  auto contemporaneamente

Ogni auto è un thread che:

1. Attende di poter entrare nel ponte.
2. Lo "attraversa" (simulato con una `sleep()`).
3. Esce, lasciando spazio a un'altra auto.

# Semafori

---

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define N_AUTO 10
#define MAX_SUL_PONTE 3

sem_t bridge; // semaforo contatore per il ponte

void* car(void* arg) {
    long id = (long)arg;
    printf("Auto %ld arriva al ponte\n", id);

    // TODO: attendere permesso di entrare (sem_wait)

    printf("Auto %ld entra nel ponte\n", id);
    sleep(1 + rand() % 2); // attraversamento
    printf("Auto %ld esce dal ponte\n", id);

    // TODO: segnalare uscita (sem_post)

    return NULL;
}
```

# Semafori

---

```
int main(void) {
    pthread_t th[N_AUTO];
    srand(time(NULL));

    // TODO: inizializzare il semaforo bridge con valore
    MAX_SUL_PONTE
    // (pshared = 0 per thread dello stesso processo)

    for (long i = 0; i < N_AUTO; i++)
        pthread_create(&th[i], NULL, car, (void*)i);

    for (int i = 0; i < N_AUTO; i++)
        pthread_join(th[i], NULL);

    // TODO: distruggere il semaforo

    return 0;
}
```

# Readers-Writers Problem

---

- Un insieme di dati è condiviso tra  $n$  processi concorrenti
  - Readers – leggono solo i dati; **non** aggiornano
  - Writers – leggono e scrivono
- **Problema** – permettere a lettori multipli di leggere allo stesso tempo
  - solo un singolo scrittore può accedere ai dati condivisi allo stesso tempo
- Diverse variazioni – basate su priorità
- Dati condivisi:
  - Data set
  - Mutex **rw\_mutex** inizializzato a 1
  - Mutex **mutex** inizializzato a 1
    - protegge **read\_count**
  - Intero **read\_count** inizializzato a 0

# Readers-Writers Problem

---

- Struttura scrittore

```
do {  
    lock(rw_mutex);  
  
    ...  
    /* writing is performed */  
    ...  
    unlock(rw_mutex);  
} while (true);
```

# Readers-Writers Problem

---

- Struttura del lettore

```
do {  
    lock(mutex);  
    read_count++;  
    if (read_count == 1)  
        lock(rw_mutex);  
    unlock(mutex);  
  
    ...  
    /* reading is performed */  
    ...  
    lock(mutex);  
    read_count--;  
    if (read_count == 0)  
        unlock(rw_mutex);  
    unlock(mutex);  
} while (true);
```

Se uno scrittore è in sezione critica con n lettori in attesa, n-1 su mutex, 1 su rw\_mutex



# Readers-Writers Problem

---

- Struttura del lettore

Primo lettore 

```
do {  
    lock(mutex);  
    read_count++;  
    if (read_count == 1)  
        lock(rw_mutex);  
    unlock(mutex);  
  
    ...  
    /* reading is performed */  
    ...  
    lock(mutex);  
    read_count--;  
    if (read_count == 0)  
        unlock(rw_mutex);  
    unlock(mutex);  
} while (true);
```

Se uno scrittore è in sezione critica con n lettori in attesa, n-1 su mutex, 1 su rw\_mutex

# Readers-Writers Problem

---

- Struttura del lettore

```
do {  
    Altri lettori in attesa  lock(mutex);  
                                read_count++;  
                                if (read_count == 1)  
                                    Primo lettore  lock(rw_mutex);  
                                unlock(mutex);  
                                ...  
                                /* reading is performed */  
                                ...  
                                lock(mutex);  
                                read_count--;  
                                if (read_count == 0)  
                                    unlock(rw_mutex);  
                                unlock(mutex);  
} while (true);
```

Se uno scrittore è in sezione critica con n lettori in attesa, n-1 su mutex, 1 su rw\_mutex

# Readers-Writers Problem

---

- Struttura del lettore

Altri lettori in attesa  `do {`

Primo lettore prende il lock di `rw_mutex` 

```
lock(mutex);
read_count++;
if (read_count == 1)
    lock(rw_mutex);
unlock(mutex);

...
/* reading is performed */
...

lock(mutex);
read_count--;
if (read_count == 0)
    unlock(rw_mutex);
unlock(mutex);
} while (true);
```

Se uno scrittore è in sezione critica con  $n$  lettori in attesa,  $n-1$  su `mutex`, 1 su `rw_mutex`

# Readers-Writers Problem

---

- Struttura del lettore

Altri lettori evitano  
il wait   
su `rw_` `rw_mutex` e  
si alternano solo su  
`mutex`

```
do {  
    lock(mutex);  
    read_count++;  
    if (read_count == 1)  
        lock(rw_mutex);  
    unlock(mutex);  
  
    ...  
    /* reading is performed */  
    ...  
    lock(mutex);  
    read_count--;  
    if (read_count == 0)  
        unlock(rw_mutex);  
    unlock(mutex);  
} while (true);
```

Se uno scrittore è in  
sezione critica con  $n$   
lettori in attesa,  $n-1$  su  
`mutex`, 1 su `rw_mutex`

# Readers-Writers Problem

---

- Struttura del lettore

```
do {  
    lock(mutex);  
    read_count++;  
    if (read_count == 1)  
        lock(rw_mutex);  
    unlock(mutex);  
  
    ...  
    /* reading is performed */  
    ...  
    lock(mutex);  
    read_count--;  
    if (read_count == 0)  
        unlock(rw_mutex);  
    unlock(mutex);  
} while (true);
```

Se uno scrittore è in sezione critica con n lettori in attesa, n-1 su mutex, 1 su rw\_mutex

Ultimo lettore  
lascia il rw\_mutex



# Readers-Writers Problem

---

- Strutture globali

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

// protegge read_count
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

// accesso esclusivo ai writer (e al primo reader)
pthread_mutex_t rw_mutex = PTHREAD_MUTEX_INITIALIZER;

// numero di lettori attualmente dentro
int read_count = 0;

// risorsa condivisa
int shared_data = 0;
```

# Readers-Writers Problem

---

- Lettore

```
void* reader(void* arg) {
    long id = (long)arg;

    while (1) {        /* --- entry section --- */
        pthread_mutex_lock(&mutex);
        read_count++;
        if (read_count == 1) { // il primo lettore blocca i writer
            pthread_mutex_lock(&rw_mutex);
        }
        pthread_mutex_unlock(&mutex);
        /* --- sezione critica (lettura) --- */
        printf("[R%d] legge: %d\n", id, shared_data);
        usleep(100000); // simuliamo il lavoro


        /* --- exit section --- */
        pthread_mutex_lock(&mutex);
        read_count--;
        if (read_count == 0) {
            // l'ultimo lettore fa uscire i writer
            pthread_mutex_unlock(&rw_mutex);
        }
        pthread_mutex_unlock(&mutex);
        usleep(100000);
    }
    return NULL;
}
```

# Readers-Writers Problem

---

- Scrittore

```
void* writer(void* arg) {
    long id = (long)arg;

    while (1) {
        pthread_mutex_lock(&rw_mutex); // ingresso esclusivo
        shared_data++;                  // scrittura
        printf(" [W%d] scrive: %d\n", id, shared_data);
        usleep(150000);                 // simuliamo il lavoro
        pthread_mutex_unlock(&rw_mutex); // uscita

        usleep(200000);
    }
    return NULL;
}
```



# Readers-Writers Problem

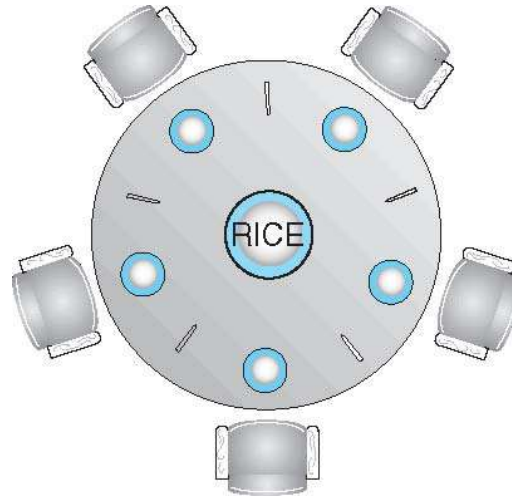
---

- Main function

```
int main(void) {  
    pthread_t r[3], w[2];  
  
    for (long i = 0; i < 3; i++)  
        pthread_create(&r[i], NULL, reader, (void*)i);  
  
    for (long i = 0; i < 2; i++)  
        pthread_create(&w[i], NULL, writer, (void*)i);  
  
    for (int i = 0; i < 3; i++)  
        pthread_join(r[i], NULL);  
    for (int i = 0; i < 2; i++)  
        pthread_join(w[i], NULL);  
  
    return 0;  
}
```

# Problema della Cena dei Filosofi

---



- Filosofi pensano e mangiano
- Non interagiscono con i vicini, a volte provano a prendere 2 bacchette (una alla volta) per mangiare
  - Due per mangiare, poi le rilasciano quando hanno finito
- Se 5 filosofi
  - Dati condivisi
    - Ciotola di riso (data set)
    - Semaforo **chopstick** [5] inizializzato a 1

# Problema della Cena dei Filosofi

---

- filosofo *i*:

```
do {
```

```
    lock (chopstick[i] );
```

```
    lock (chopStick[ (i + 1) % 5] );
```

```
        // eat
```

```
    unlock (chopstick[i] );
```

```
    unlock (chopstick[ (i + 1) % 5] );
```

```
        // think
```

```
    } while (TRUE);
```

- Problema di questo algoritmo?

# Soluzione con Monitor

---

```
monitor DiningPhilosophers
```

```
{
```

```
    enum { THINKING, HUNGRY, EATING} state [5] ;
```

```
    condition self [5];
```

stati usati per  
coordinare le azioni

```
    void pickup (int i) {
```

```
        state[i] = HUNGRY;
```

```
        test(i);
```

```
        if (state[i] != EATING) self[i].wait;
```

test valuta e cambia  
stato in EATING

```
}
```

Self[i] Permette ad *i* di  
ritardare quando è  
affamato ma non può  
prendere entrambe le  
bacchette.

```
    void putdown (int i) {
```

```
        state[i] = THINKING;
```

```
        // test left and right neighbors
```

```
        test((i + 4) % 5);
```

```
        test((i + 1) % 5);
```

```
}
```

Un filosofo alla volta mangia (quando ha fame) prendendo entrambe le bacchette. Se ha fame controlla prima se ci sono entrambe le bacchette.

# Soluzione con Monitor

---

```
void test (int i) {
    if ((state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING) ) {
        state[i] = EATING ;
        self[i].signal () ;
    }
}

initialization_code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
}
```

Verifica se i vicini di i mangiano nel caso sia affamato. Se non mangiano si setta in EATING. Dopo il controllo sblocca i

# Soluzione con Monitor

---

- Ogni filosofo *i* invoca `pickup()` e `putdown()`

`DiningPhilosophers.pickup(i);`

`EAT`

`DiningPhilosophers.putdown(i);`

- Non deadlock, ma starvation è possibile

# Soluzione con CondVar

---

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define N 5
#define LEFT(i)  ((i + N - 1) % N)
#define RIGHT(i) ((i + 1) % N)

typedef enum { THINKING, HUNGRY, EATING } state_t;

/* --- Variabili globali --- */
state_t state[N];           // stato di ciascun filosofo
pthread_mutex_t mtx;        // mutex del monitor
pthread_cond_t self[N];     // condizione associata a ogni
                             filosofo

/* --- Prototipi funzioni del monitor --- */
void pickup(int i);
void putdown(int i);
void test(int i);
```

# Soluzione con CondVar

---

```
int main(void) {
    pthread_t th[N];

    pthread_mutex_init(&mtx, NULL);
    for (int i = 0; i < N; i++) {
        state[i] = THINKING;
        pthread_cond_init(&self[i], NULL);
    }

    for (long i = 0; i < N; i++) {
        if (pthread_create(&th[i], NULL, philosopher, (void*)i) != 0) {
            perror("pthread_create");
            exit(1);
        }
    }

    for (int i = 0; i < N; i++)
        pthread_join(th[i], NULL);

    pthread_mutex_destroy(&mtx);
    for (int i = 0; i < N; i++)
        pthread_cond_destroy(&self[i]);

    return 0;
}
```



# Soluzione con CondVar

---

```
/* --- Thread dei filosofi --- */

void* philosopher(void* arg) {
    int i = (int)(long)arg;

    while (1) {
        printf("Filosofo %d sta PENSANDO\n", i);
        usleep(100000 + rand() % 200000);

        pickup(i);

        printf(">>> Filosofo %d sta MANGIANDO\n", i);
        usleep(100000 + rand() % 200000);

        putdown(i);
    }
    return NULL;
}
```

# Soluzione con CondVar

---

```
void test(int i) {
    // posso mangiare se:
    // - io sono HUNGRY
    // - il vicino di sinistra NON sta mangiando
    // - il vicino di destra NON sta mangiando
    if (state[i] == HUNGRY &&
        state[LEFT(i)] != EATING &&
        state[RIGHT(i)] != EATING) {

        state[i] = EATING;
        // sveglia proprio questo filosofo (se era in wait)
        pthread_cond_signal(&self[i]);
    }
}
```

# Soluzione con CondVar

---

```
void pickup(int i) {
    pthread_mutex_lock(&mtx);

    // dichiaro di avere fame
    state[i] = HUNGRY;

    // provo a mangiare subito
    test(i);

    // se non sono riuscito a entrare in EATING, aspetto
    while (state[i] != EATING)
        pthread_cond_wait(&self[i], &mtx);

    pthread_mutex_unlock(&mtx);
}
```

# Soluzione con CondVar

---

```
void putdown(int i) {  
    pthread_mutex_lock(&mtx);  
  
    // ho finito di mangiare → torno THINKING  
    state[i] = THINKING;  
  
    // dopo che ho liberato le bacchette,  
    // provo a far mangiare i vicini  
    test(LEFT(i));  
    test(RIGHT(i));  
  
    pthread_mutex_unlock(&mtx);  
}
```